Семинар 07. Self-Supervised Vision Models

Занятие провел: Оганов Александр (tg: @welmud)

Материалы составил: Оганов Александр (tg: @welmud)

Москва, Осенний семестр 2026

Источники:

- **DINO**: Mathilde Caron et al., 2021, примерно 13000 цитирований
- **DINOv2**: Maxime Oquab et al., 2023, примерно 13000 цитирований
- **DINOv3**: Oriane Siméoni et al., 2025, примерно 1600 цитирований
- **Averaging Weights Leads to Wider Optima and Better Generalization**: Pavel Izmailov et al., 2018, примерно 2700 цитирований
- GitHub реализация DINOv1
- GitHub реализация DINOv2
- GitHub реализация DINOv3

In [1]:
```
%pip install torchmetrics -q
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/983.4 kB ? eta -:--:--
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 983.0/983.4 kB 45.9 MB/s eta 0:0
0:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 983.4/983.4 kB 17.2 MB/s eta 0:0
0:00
```

In [2]:
```
import os
```

In [3]:
```
if not os.path.exists("dinov3"):
  !git clone https://github.com/facebookresearch/dinov3.git
```

```
Cloning into 'dinov3'...
remote: Enumerating objects: 660, done.
remote: Counting objects: 100% (328/328), done.
remote: Compressing objects: 100% (173/173), done.
remote: Total 660 (delta 195), reused 155 (delta 155), pack-reused 332 (from
1)
Receiving objects: 100% (660/660), 12.99 MiB | 19.64 MiB/s, done.
Resolving deltas: 100% (269/269), done.
```

In [4]:
```python
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

In [5]:
```python
import pickle
import os
import io
import tarfile
import urllib

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import ConnectionPatch
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn.functional as F
import torchvision.transforms.functional as TF

from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from scipy import signal
```

In [6]:
```python
REPO_DIR = "./dinov3"
WEIGHTS_DIR = "/content/drive/MyDrive/DINOv3_seminar/dinov3_vitb16_pretrain_

# DINOv3 ViT models pretrained on web images)
model = torch.hub.load(REPO_DIR, 'dinov3_vitb16', source='local', weights=WE
model.cuda()
model.eval()
```

```
Downloading: "file:///content/drive/MyDrive/DINOv3_seminar/dinov3_vitb16_pre
train_lvd1689m-73cec8be.pth" to /root/.cache/torch/hub/checkpoints/dinov3_vi
tb16_pretrain_lvd1689m-73cec8be.pth
100%|██████████| 327M/327M [00:09<00:00, 37.9MB/s]
```

Out[6]:
```
DinoVisionTransformer(
  (patch_embed): PatchEmbed(
    (proj): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
    (norm): Identity()
  )
  (rope_embed): RopePositionEmbedding()
  (blocks): ModuleList(
    (0-11): 12 x SelfAttentionBlock(
      (norm1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (attn): SelfAttention(
        (qkv): LinearKMaskedBias(in_features=768, out_features=2304, bias=T
rue)
        (attn_drop): Dropout(p=0.0, inplace=False)
        (proj): Linear(in_features=768, out_features=768, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
      )
      (ls1): LayerScale()
      (norm2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (mlp): Mlp(
        (fc1): Linear(in_features=768, out_features=3072, bias=True)
        (act): GELU(approximate='none')
        (fc2): Linear(in_features=3072, out_features=768, bias=True)
        (drop): Dropout(p=0.0, inplace=False)
      )
      (ls2): LayerScale()
    )
  )
  (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (head): Identity()
)
```

In [7]:
```
print(f"Всего {sum(p.numel() for p in model.parameters())} параметров")
```

```
Всего 85669632 параметров
```

In [8]:
```
from transformers import AutoImageProcessor, AutoModel, TorchAoConfig
from transformers.image_utils import load_image


url = "http://images.cocodataset.org/val2017/000000039769.jpg"
image = load_image(url)
print("Изображение размера:", image.size)
image
```

```
Изображение размера: (640, 480)
```

Out[8]:



Мы будем работать с патчем размера 16 и изображения размера 768.

In [9]:
```
PATCH_SIZE = 16
IMAGE_SIZE = 768

IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)
```

In [10]:
```
def resize_transform(
    mask_image: Image,
    image_size: int = IMAGE_SIZE,
    patch_size: int = PATCH_SIZE,
) -> torch.Tensor:
    w, h = mask_image.size
    h_patches = int(image_size / patch_size)
    w_patches = int((w * image_size) / (h * patch_size))
    return TF.to_tensor(TF.resize(mask_image, (h_patches * patch_size, w_pat

image_resized = resize_transform(image)
image_resized.shape
```

Out[10]:
```
torch.Size([3, 768, 1024])
```

Соберем признаки со всех промежуточных слоев, всего слоев 12.

In [11]:
```
@torch.inference_mode()
def get_features(image_resized):
```

```
  n_layers = 12
  image_resized_norm = TF.normalize(image_resized, mean=IMAGENET_MEAN, std=I
  img_obj = image_resized_norm[None].cuda()
  with torch.autocast(device_type='cuda', dtype=torch.float32):
    return model.get_intermediate_layers(img_obj, n=range(n_layers), reshape
```

In [12]:
```
output = get_features(image_resized)
print(f"Мы собрали признаки с {len(output)} слоёв")
```

```
Мы собрали признаки с 12 слоёв
```

In [13]:
```
last_features = output[-1].squeeze().detach().cpu()
last_features.shape
```

Out[13]:
```
torch.Size([768, 48, 64])
```

Попробуем визулизировать фичи. 768 – размерность эмбеддинга каждого патча, всего патчей 48×64. Для визуализации понизим размерность до 3.

In [14]:
```
def features_to_matrix(features):
  x = features.clone()
  return x.view(x.shape[0], -1).permute(1, 0)

x = features_to_matrix(last_features)
x.shape
```

Out[14]:
```
torch.Size([3072, 768])
```

In [15]:
```
pca = PCA(n_components=3, whiten=True, random_state=42)
x_transform = pca.fit_transform(x)

x_transform.shape, type(x_transform)
```

Out[15]:
```
((3072, 3), numpy.ndarray)
```

Вернемся к исходным размерам.

In [16]:
```
h_patches, w_patches = [int(d / PATCH_SIZE) for d in image_resized.shape[1:]
```

In [17]:
```
def vis_img_and_dense(image_resized, projected_image):
  # multiply by 2.0 and pass through a sigmoid to get vibrant colors
  projected_image = torch.nn.functional.sigmoid(projected_image.mul(2.0)).pe

  fig, ax = plt.subplots(1, 2, figsize=(10, 5))
  ax[0].imshow(image_resized.permute(1, 2, 0))
  ax[1].imshow(projected_image.permute(1, 2, 0))
  ax[0].set_title("Изображение", fontsize=15)
  ax[1].set_title("Внутреннее представление", fontsize=15)

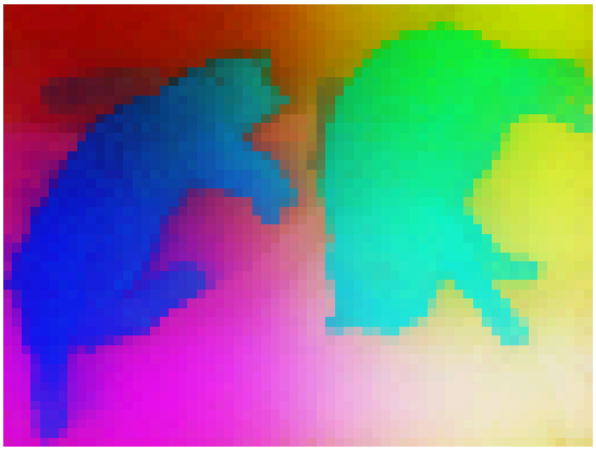
  for a in ax:
    a.axis('off')
  plt.tight_layout()
  plt.show()
```

```
projected_image = torch.from_numpy(x_transform).view(h_patches, w_patches, 3
vis_img_and_dense(image_resized, projected_image)
```

Изображение



Внутреннее представление



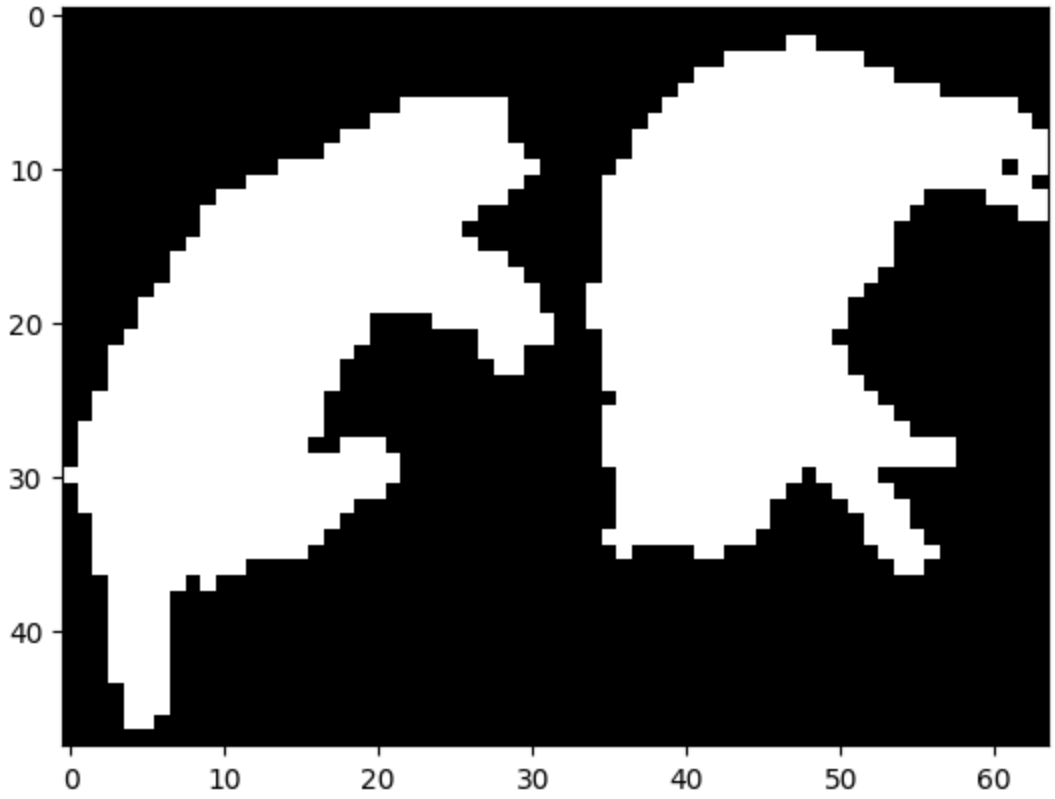
Обучим на признаках K-Means

In [18]:
```
def return_kmeans_bin(x, h_patches, w_patches):
  kmeans = KMeans(n_clusters=2, random_state=42)
  kmeans_labels = kmeans.fit_predict(x)
  print(np.unique(kmeans_labels, return_counts=True))

  return kmeans_labels.reshape(h_patches, w_patches)
```

In [19]:
```
kmeans_labels = return_kmeans_bin(x, h_patches, w_patches)
plt.imshow(kmeans_labels, cmap='gray')
plt.show()
```

```
(array([0, 1], dtype=int32), array([1876, 1196]))
```



In [20]:
```
projected_image = torch.from_numpy(x_transform).view(h_patches, w_patches, 3
kmeans_labels_torch = torch.from_numpy(kmeans_labels)[:, :, None]
masked = torch.where(kmeans_labels_torch == 1, projected_image, -torch.inf)
vis_img_and_dense(image_resized, masked)
```



Посмотрим на какое-нибудь тестовое изображение.

Источник

In [21]:
```
image = Image.open("/content/drive/MyDrive/DINOv3_seminar/cat_warrior.jpeg")
print(image.size)
image.resize((512, 256))
```

```
(1280, 640)
```

Out[21]:



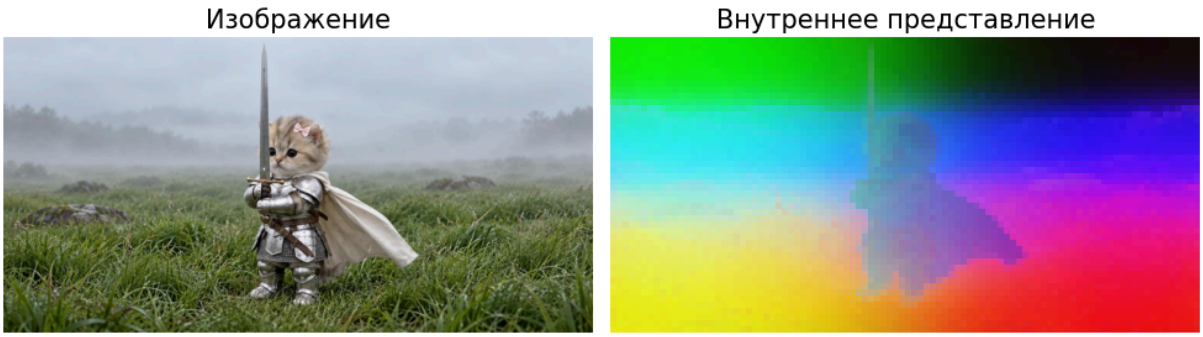
In [22]:
```
image_resized = resize_transform(image)
output = get_features(image_resized)

last_features = output[-1].squeeze().detach().cpu()
x = features_to_matrix(last_features)

pca = PCA(n_components=3, whiten=True, random_state=42)
x_transform = pca.fit_transform(x)

h_patches, w_patches = [int(d / PATCH_SIZE) for d in image_resized.shape[1:]
```

In [23]:
```
projected_image = torch.from_numpy(x_transform).view(h_patches, w_patches, 3
vis_img_and_dense(image_resized, projected_image)
```



Сегментация

Источник

In [24]:
```
IMAGES_URI = "https://dl.fbaipublicfiles.com/dinov3/notebooks/foreground_seg
LABELS_URI = "https://dl.fbaipublicfiles.com/dinov3/notebooks/foreground_seg

def load_images_from_remote_tar(tar_uri: str) -> list[Image.Image]:
    images = []
    with urllib.request.urlopen(tar_uri) as f:
        tar = tarfile.open(fileobj=io.BytesIO(f.read()))
        for member in tar.getmembers():
            image_data = tar.extractfile(member)
```

```
            image = Image.open(image_data)
            images.append(image)
    return images

images = load_images_from_remote_tar(IMAGES_URI)
labels = load_images_from_remote_tar(LABELS_URI)
n_images = len(images)
assert n_images == len(labels), f"{len(images)=}, {len(labels)=}"

print(f"Loaded {n_images} images and labels")
```

```
Loaded 9 images and labels
```

In [25]:
```
data_index = 0

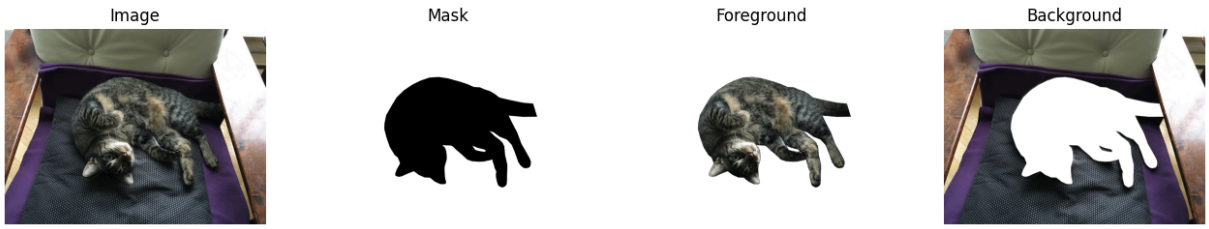
print(f"Showing image / mask at index {data_index}:")

image = images[data_index]
mask = labels[data_index]
foreground = Image.composite(image, mask, mask)
mask_bg_np = np.copy(np.array(mask))
mask_bg_np[:, :, 3] = 255 - mask_bg_np[:, :, 3]
mask_bg = Image.fromarray(mask_bg_np)
background = Image.composite(image, mask_bg, mask_bg)

data_to_show = [image, mask, foreground, background]
data_labels = ["Image", "Mask", "Foreground", "Background"]

plt.figure(figsize=(16, 4))
for i in range(len(data_to_show)):
    plt.subplot(1, len(data_to_show), i + 1)
    plt.imshow(data_to_show[i])
    plt.axis('off')
    plt.title(data_labels[i], fontsize=12)
```

```
Showing image / mask at index 0:
```



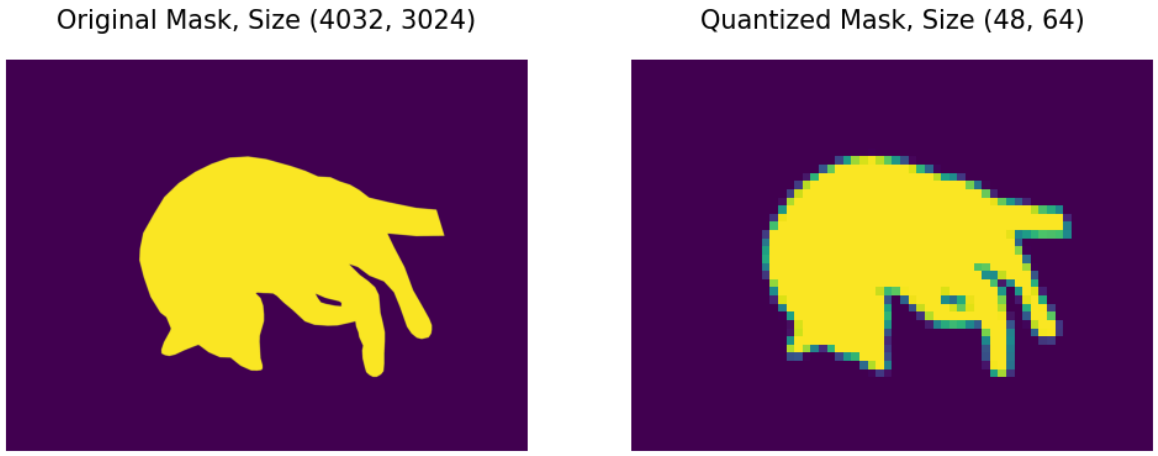
Нам нужно будет перевести маски в соотвествующий размеры.

In [26]:
```
patch_quant_filter = torch.nn.Conv2d(1, 1, PATCH_SIZE, stride=PATCH_SIZE, bi
patch_quant_filter.weight.data.fill_(1.0 / (PATCH_SIZE * PATCH_SIZE))

mask_0 = labels[0].split()[-1]
mask_0_resized = resize_transform(mask_0)
with torch.no_grad():
    mask_0_quantized = patch_quant_filter(mask_0_resized).squeeze().detach()

plt.figure(figsize=(4, 2), dpi=300)
plt.subplot(1, 2, 1)
```

```
plt.imshow(mask_0)
plt.axis('off')
plt.title(f"Original Mask, Size {mask_0.size}", fontsize=5)
plt.subplot(1, 2, 2)
plt.imshow(mask_0_quantized)
plt.axis('off')
plt.title(f"Quantized Mask, Size {tuple(mask_0_quantized.shape)}", fontsize=
plt.show()
```



Соберем активации и приведем таргеты в нужные размеры.

In [27]:
```
xs = []
ys = []
image_index = []


with torch.inference_mode():
    with torch.autocast(device_type='cuda', dtype=torch.float32):
        for i in tqdm(range(n_images), desc="Processing images"):
            # Loading the ground truth
            mask_i = labels[i].split()[-1]
            mask_i_resized = resize_transform(mask_i)
            mask_i_quantized = patch_quant_filter(mask_i_resized).squeeze().
            ys.append(mask_i_quantized)
            # Loading the image data
            image_i = images[i].convert('RGB')
            image_i_resized = resize_transform(image_i)

            feats = get_features(image_i_resized)
            dim = feats[-1].shape[1]
            xs.append(feats[-1].squeeze().view(dim, -1).permute(1,0).detach(

            image_index.append(i * torch.ones(ys[-1].shape))


# Concatenate all lists into torch tensors
xs_cat = torch.cat(xs)
ys_cat = torch.cat(ys)
image_index = torch.cat(image_index)
```

```
# keeping only the patches that have clear positive or negative label
idx = (ys_cat < 0.01) | (ys_cat > 0.99)
xs_cat = xs_cat[idx]
ys_cat = ys_cat[idx]
image_index = image_index[idx]

print("Design matrix of size : ", xs_cat.shape)
print("Label matrix of size : ", ys_cat.shape)
```

```
Processing images: 100%|██████████| 9/9 [00:05<00:00,  1.60it/s]
Design matrix of size :  torch.Size([23604, 768])
Label matrix of size :  torch.Size([23604])
```

Посмотрим как справляется KMeans для некоторых изображений.

In [28]:
```
for id in [0, 1, 2]:
  img_resized = resize_transform(images[id].convert('RGB'))
  h_patches, w_patches = [int(d / PATCH_SIZE) for d in img_resized.shape[1:]]

  kmeans_labels = return_kmeans_bin(xs[id], h_patches, w_patches)

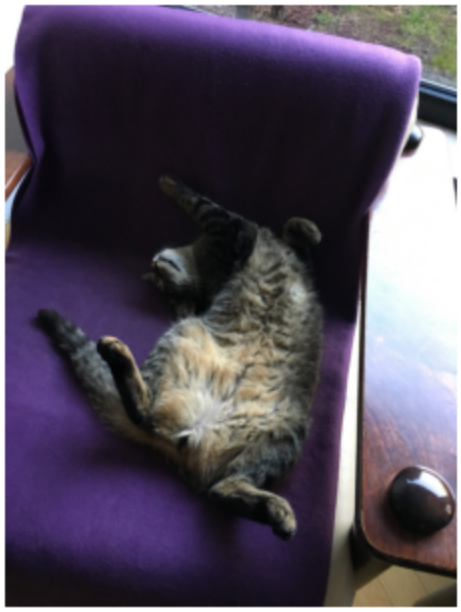
  plt.subplot(1, 2, 1)
  plt.imshow(img_resized.permute(1, 2, 0))
  plt.axis('off')
  plt.subplot(1, 2, 2)
  plt.imshow(kmeans_labels, cmap='gray')
  plt.axis('off')
  plt.show()
```

```
(array([0, 1], dtype=int32), array([1183, 1889]))
```





```
(array([0, 1], dtype=int32), array([ 655, 1073]))
```




```
(array([0, 1], dtype=int32), array([1843, 1229]))
```




Теперь мы можем обучить линейную модель.

In [29]:
```
clf = LogisticRegression(random_state=0, C=0.1, max_iter=100000, verbose=0)
clf.fit(xs_cat.numpy(), (ys_cat > 0).long().numpy())
```

Out[29]:

LogisticRegression

```
LogisticRegression(C=0.1, max_iter=100000, random_state=0)
```

Проверим качество на тестовом изображении.

In [30]:
```
test_image_fpath = "https://dl.fbaipublicfiles.com/dinov3/notebooks/foregrou
test_image = load_image(test_image_fpath)
test_image_resized = resize_transform(test_image)
plt.imshow(test_image_resized.permute(1, 2, 0))
plt.axis('off')
plt.show()
```



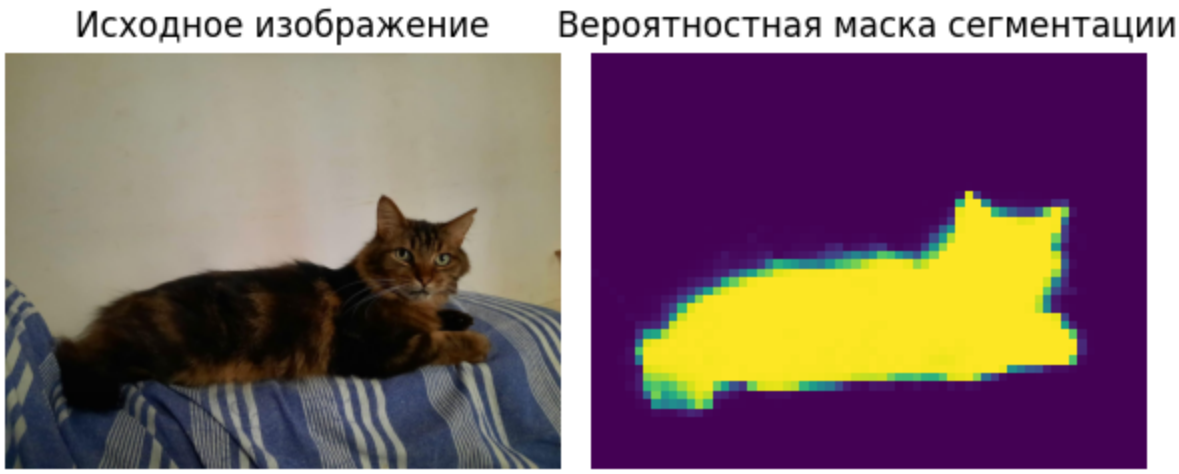
In [31]:
```
feats = get_features(test_image_resized)
x = feats[-1].squeeze().detach().cpu()
dim = x.shape[0]
x = x.view(dim, -1).permute(1, 0)
```

In [32]:
```
def vis_segm(test_image_resized, x):

  h_patches, w_patches = [int(d / PATCH_SIZE) for d in test_image_resized.sh

  fg_score = clf.predict_proba(x)[:, 1].reshape(h_patches, w_patches)

  plt.figure(figsize=(6, 3))
  plt.subplot(1, 2, 1)
  plt.axis('off')
  plt.imshow(test_image_resized.permute(1, 2, 0))
  plt.title('Исходное изображение')
  plt.subplot(1, 2, 2)
  plt.axis('off')
  plt.imshow(fg_score)
  plt.title('Вероятностная маска сегментации')
  plt.tight_layout()
  plt.show()
vis_segm(test_image_resized, x)
```

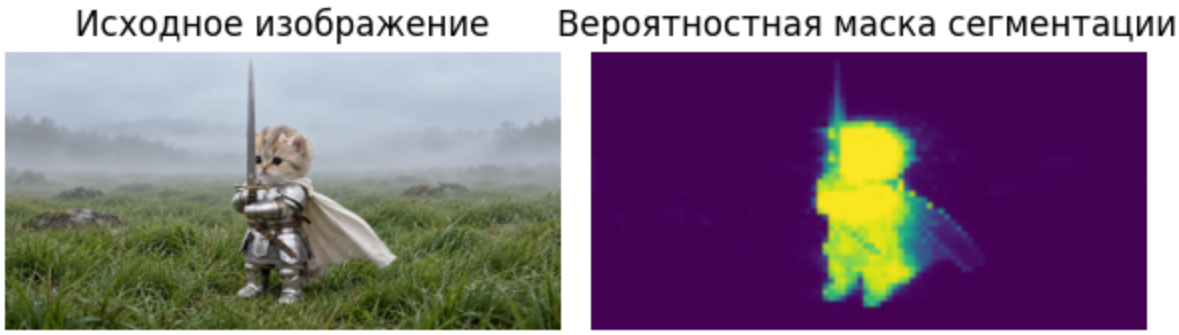


Посмотрим как метод обобщается на другие домены.

In [33]:
```
test_image = Image.open("/content/drive/MyDrive/DINOv3_seminar/cat_warrior.j
test_image_resized = resize_transform(test_image)

feats = get_features(test_image_resized)
x = feats[-1].squeeze().detach().cpu()
dim = x.shape[0]
x = x.view(dim, -1).permute(1, 0)
```

In [34]:
```
vis_segm(test_image_resized, x)
```



In [35]:
```
save_root = '.'
model_path = os.path.join(save_root, "fg_classifier.pkl")
with open(model_path, "wb") as f:
  pickle.dump(clf, f)
```

Сапоставление двух изображений

Источник

In [36]:
```
image_left_uri = "https://dl.fbaipublicfiles.com/dinov3/notebooks/dense_spar
mask_left_uri = "https://dl.fbaipublicfiles.com/dinov3/notebooks/dense_spars
image_right_uri = "https://dl.fbaipublicfiles.com/dinov3/notebooks/dense_spa
mask_right_uri = "https://dl.fbaipublicfiles.com/dinov3/notebooks/dense_spar
```

```
def load_image_from_url(url: str) -> Image:
    with urllib.request.urlopen(url) as f:
        return Image.open(f)


image_left = load_image_from_url(image_left_uri)
mask_left = load_image_from_url(mask_left_uri)

image_right = load_image_from_url(image_right_uri)
mask_right = load_image_from_url(mask_right_uri)
```

In [37]:

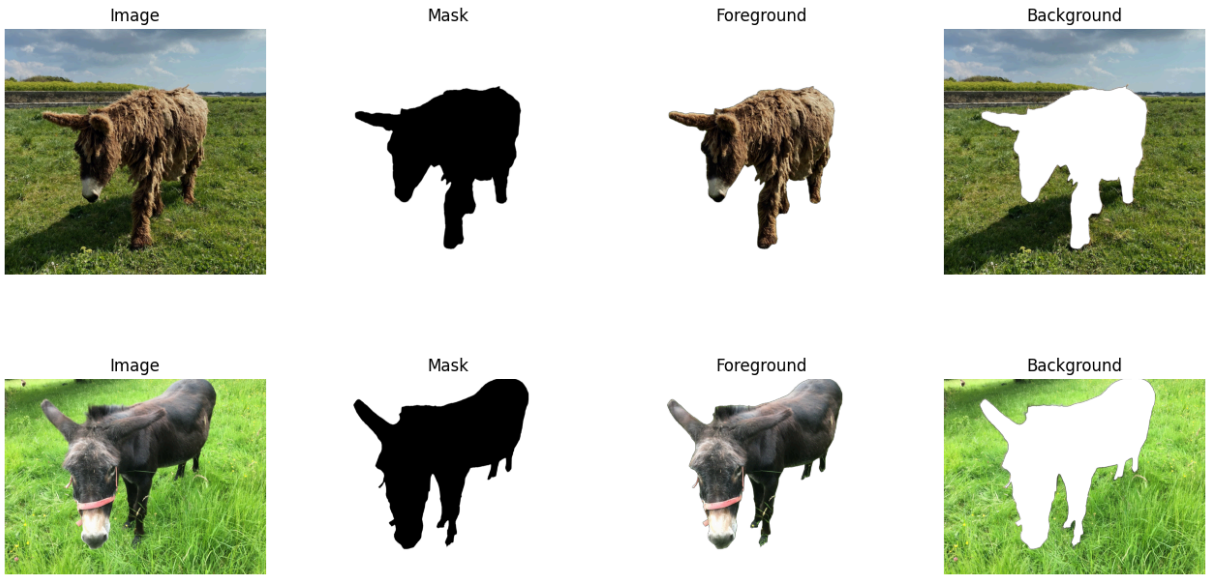
```
plt.figure(figsize=(16, 8))

for j, (image, mask) in enumerate([(image_left, mask_left), (image_right, ma
    foreground = Image.composite(image, mask, mask)
    mask_bg_np = np.copy(np.array(mask))
    mask_bg_np[:, :, 3] = 255 - mask_bg_np[:, :, 3]
    mask_bg = Image.fromarray(mask_bg_np)
    background = Image.composite(image, mask_bg, mask_bg)

    data_to_show = [image, mask, foreground, background]
    data_labels = ["Image", "Mask", "Foreground", "Background"]


    for i in range(len(data_to_show)):
        plt.subplot(2, len(data_to_show), 4 * j + i + 1)
        plt.imshow(data_to_show[i])
        plt.axis('off')
        plt.title(data_labels[i], fontsize=12)

plt.show()
```



In [38]:

```
def vis_raw_imgs(image_left, image_right):
  fig = plt.figure(figsize=(4, 2), dpi=300)
  ax1 = fig.add_subplot(121)
  ax1.imshow(image_left)
  ax1.set_axis_off()
  ax1.set_title("Изображение слева", fontsize=5)
```

```
  ax2 = fig.add_subplot(122)
  ax2.imshow(image_right)
  ax2.set_axis_off()
  ax2.set_title("Изображение справа", fontsize=5)

  plt.show()
```

In [39]:
```
vis_raw_imgs(image_left, image_right)
```



In [40]:
```
patch_mask_values = []
patch_features = []

with torch.inference_mode():
    with torch.autocast(device_type='cuda', dtype=torch.float32):
        for image, mask in tqdm([(image_left, mask_left), (image_right, mask
            # processing mask
            mask = mask.split()[-1]
            mask_resized = resize_transform(mask)
            #mask_quantized = patch_quant_filter(mask_resized).squeeze().vie
            mask_quantized = patch_quant_filter(mask_resized).squeeze().deta
            patch_mask_values.append(mask_quantized)
            # processing image
            image = image.convert('RGB')
            image_resized = resize_transform(image)

            feats = get_features(image_resized)
            dim = feats[-1].shape[1]
            patch_features.append(feats[-1].squeeze().detach().cpu())
```

```
Processing images: 100%|██████████| 2/2 [00:00<00:00,  2.56it/s]
```

In [41]:
```
MASK_FG_THRESHOLD = 0.5

patch_features[0] = F.normalize(patch_features[0], p=2, dim=0)
patch_features[1] = F.normalize(patch_features[1], p=2, dim=0)

heatmaps = torch.einsum(
```

```
    "k f, f h w -> k h w",
    patch_features[0].view(dim, -1).permute(1, 0),
    patch_features[1],
)

# compute 2D patch locations in the left image
n_patches_left = patch_features[0].shape[1] * patch_features[0].shape[2]
patch_indices_left = torch.arange(n_patches_left)
locs_2d_left = (
    torch.stack(
        (
            patch_indices_left // patch_features[0].shape[2],  # row
            patch_indices_left % patch_features[0].shape[2]    # column
        ),
        dim=-1
    ) + 0.5
) * PATCH_SIZE

# compute the corresponding 2D patch locations in the right image
patch_indices_right = torch.flatten(heatmaps, start_dim=-2).argmax(dim=-1)
locs_2d_right = (
    torch.stack(
        (
            patch_indices_right // patch_features[1].shape[2],  # row
            patch_indices_right % patch_features[1].shape[2]    # column
        ),
        dim=-1
    ) + 0.5
) * PATCH_SIZE

# foreground patches mask in the left image
patches_left_fg_selection = (patch_mask_values[0].view(-1) > MASK_FG_THRESHO
# left image patches mask for patches that map to a foreground patch in the
patches_right_fg_selection = (patch_mask_values[1].view(-1)[patch_indices_ri
# select foreground left image patches that map to foreground right image pa
patches_fg_selection = (patches_left_fg_selection * patches_right_fg_selecti

# select (row, col) coordinates for the matched patches
locs_2d_left_fg = locs_2d_left[patches_fg_selection, :]
locs_2d_right_fg = locs_2d_right[patches_fg_selection, :]
```

In [42]:
```
pca = PCA(n_components=3, whiten=True)
fg_patches_left = features_to_matrix(patch_features[0])[patches_fg_selectior
pca.fit(fg_patches_left)
```

Out[42]:

PCA

```
PCA(n_components=3, whiten=True)
```

In [43]:
```
def vis_dense_img(patch_features):
  # get colors for the left image
  h_patches_left = patch_features[0].shape[1]
  w_patches_left = patch_features[0].shape[2]
  x_left = patch_features[0].view(dim, -1).permute(1, 0)
```

```
  projected_image_left = torch.from_numpy(
      pca.transform(x_left.numpy())
  ).view(h_patches_left, w_patches_left, 3)
  # multiply by 2.0 and pass through a sigmoid to get vibrant colors
  projected_image_left = torch.nn.functional.sigmoid(projected_image_left.mu

  # get colors for the right image
  h_patches_right = patch_features[1].shape[1]
  w_patches_right = patch_features[1].shape[2]
  x_right = patch_features[1].view(dim, -1).permute(1, 0)
  projected_image_right = torch.from_numpy(
      pca.transform(x_right.numpy())
  ).view(h_patches_right, w_patches_right, 3)
  projected_image_right = torch.nn.functional.sigmoid(projected_image_right.

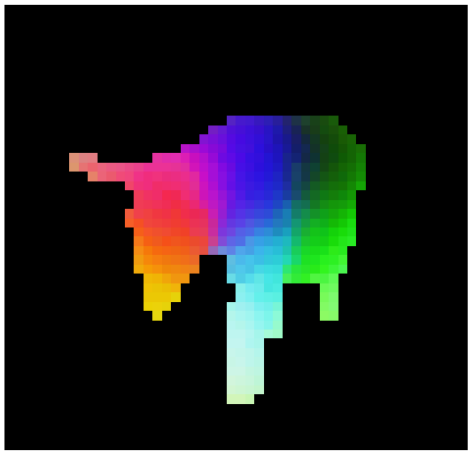
  # apply foreground masks to both visualizations
  projected_image_left *= (patch_mask_values[0] > MASK_FG_THRESHOLD)[None, :
  projected_image_right *= (patch_mask_values[1] > MASK_FG_THRESHOLD)[None,

  plt.figure(figsize=(4, 2), dpi=300)
  plt.subplot(1, 2, 1)
  plt.imshow(projected_image_left.permute(1, 2, 0))
  plt.title("Изображение слева, Внутреннее представление", fontsize=5)
  plt.axis('off')
  plt.subplot(1, 2, 2)
  plt.imshow(projected_image_right.permute(1, 2, 0))
  plt.title("Изображение справа, Внутреннее представление", fontsize=5)
  plt.axis('off')
  plt.show()

  return projected_image_left

projected_image_left = vis_dense_img(patch_features)
```

Изображение слева, Внутреннее представление



Изображение справа, Внутреннее представление



In [44]:

```
def compute_distances_l2(X, Y, X_squared_norm, Y_squared_norm):
    distances = -2 * X @ Y.T
    distances.add_(X_squared_norm[:, None]).add_(Y_squared_norm[None, :])
    return distances
```

```
def stratify_points(pts_2d: torch.Tensor, threshold: float = 100.0) -> torch
    # pts_2d: [N, 2]
    n = len(pts_2d)
    max_value = threshold + 1
    pts_2d_sq_norms = torch.linalg.vector_norm(pts_2d, dim=1)
    pts_2d_sq_norms.square_()
    distances = compute_distances_l2(pts_2d, pts_2d, pts_2d_sq_norms, pts_2d
    distances.fill_diagonal_(max_value)
    distances_mask = torch.empty((n, n), dtype=pts_2d.dtype, device=pts_2d.d
    torch.le(distances, threshold, out=distances_mask)
    ones_vec = torch.ones(n, device=pts_2d.device, dtype=pts_2d.dtype)
    counts_vec = torch.mv(distances_mask, ones_vec)
    indices_mask = np.ones(n)
    while torch.any(counts_vec).item():
        index_max = torch.argmax(counts_vec).item()
        indices_mask[index_max] = 0
        distances[index_max, :] = max_value
        distances[:, index_max] = max_value
        torch.le(distances, threshold, out=distances_mask)
        torch.mv(distances_mask, ones_vec, out=counts_vec)
    indices_to_exclude = np.nonzero(indices_mask == 0)[0]
    indices_to_keep = np.nonzero(indices_mask > 0)[0]
    return indices_to_exclude, indices_to_keep


# image scale to go from patches to the original image coordinates
scale_left = image_left.height / IMAGE_SIZE
scale_right = image_right.height / IMAGE_SIZE

STRATIFY_DISTANCE_THRESHOLD = 100.0

print(f"Non-stratified points: {tuple(locs_2d_left_fg.shape)}")

indices_to_exclude, indices_to_keep = stratify_points(locs_2d_left_fg * scal

sparse_points_left_yx = locs_2d_left_fg[indices_to_keep, :].cpu().numpy()
sparse_points_right_yx = locs_2d_right_fg[indices_to_keep, :].cpu().numpy()

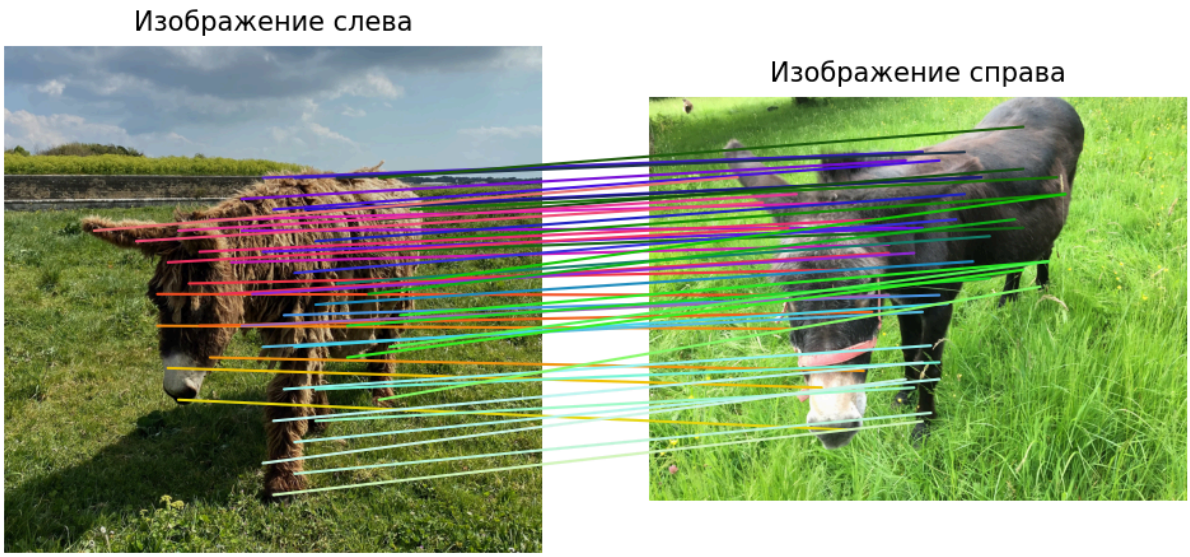
print(f"Stratified points: {sparse_points_left_yx.shape}")
```

```
Non-stratified points: (476, 2)
Stratified points: (53, 2)
```

In [45]:
```
# show original left and right images
def show_points(image_left, image_right, indices_to_keep, sparse_points_left
  fig = plt.figure(figsize=(8,4), dpi=150)
  ax1 = fig.add_subplot(121)
  ax1.imshow(image_left)
  ax1.set_axis_off()
  ax1.set_title("Изображение слева", fontsize=10)
  ax2 = fig.add_subplot(122)
  ax2.imshow(image_right)
  ax2.set_axis_off()
  ax2.set_title("Изображение справа", fontsize=10)
```

```
    for i, (row_left, col_left), (row_right, col_right) in zip(
        indices_to_keep, sparse_points_left_yx, sparse_points_right_yx
    ):
        row_left_orig, col_left_orig = locs_2d_left_fg[i]
        # use the color used for PCA visualization
        color = projected_image_left[
            :,
            int(row_left_orig / PATCH_SIZE),
            int(col_left_orig / PATCH_SIZE)
        ].cpu().numpy()
        con = ConnectionPatch(
            xyA=(col_left * scale_left, row_left * scale_left),
            xyB=(col_right * scale_right, row_right * scale_right),
            coordsA="data",
            coordsB="data",
            axesA=ax1,
            axesB=ax2,
            color=color,
        )
        ax2.add_artist(con)
```

In [46]:
```
show_points(image_left, image_right, indices_to_keep, sparse_points_left_yx,
```



Мы можем поменять `STRATIFY_DISTANCE_THRESHOLD`, чтобы взять меньше сапоставлений.

In [47]:
```
STRATIFY_DISTANCE_THRESHOLD = 200.0

print(f"Non-stratified points: {tuple(locs_2d_left_fg.shape)}")

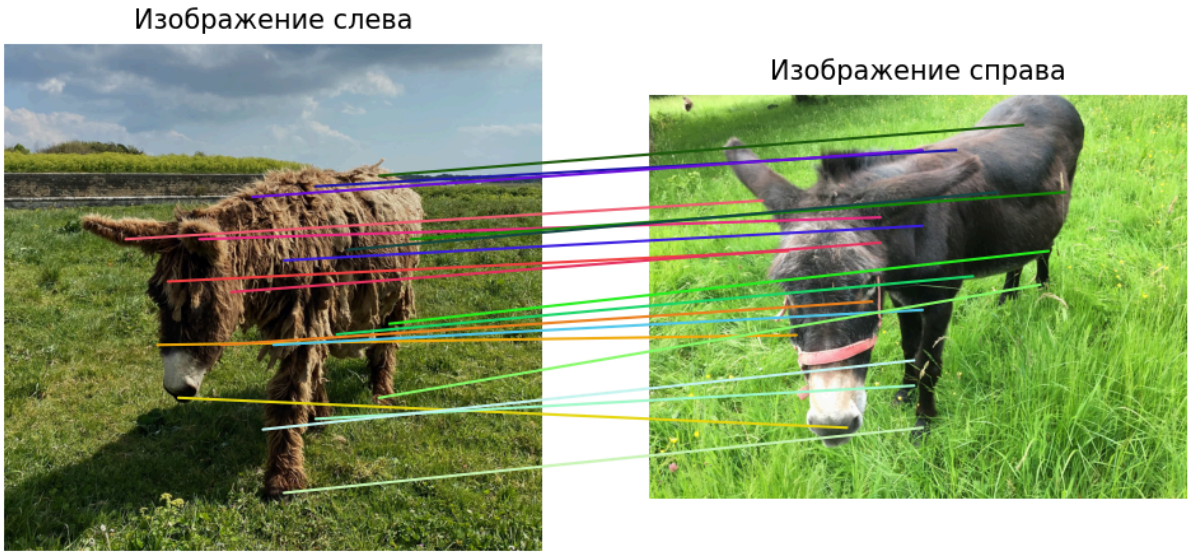
indices_to_exclude, indices_to_keep = stratify_points(locs_2d_left_fg * scal

sparse_points_left_yx = locs_2d_left_fg[indices_to_keep, :].cpu().numpy()
sparse_points_right_yx = locs_2d_right_fg[indices_to_keep, :].cpu().numpy()

print(f"Stratified points: {sparse_points_left_yx.shape}")
```

```
Non-stratified points: (476, 2)
Stratified points: (20, 2)
```

In [48]:
```
show_points(image_left, image_right, indices_to_keep, sparse_points_left_yx,
```



Проверим на изображении из другого домена.

Источник

In [49]:
```
image_right = Image.open("/content/drive/MyDrive/DINOv3_seminar/ANDA.webp")
print(image_right.size)
image_right.resize((100, 125))
```

```
(1000, 1250)
```

Out[49]:



Нам потребуется маска изображения.

In [50]:
```
mask_right = image_right.getchannel("A")
mask_right.resize((100, 125))
```

Out[50]:



In [51]:
```
vis_raw_imgs(image_left, image_right)
```

Изображение слева



Изображение справа



Повторим код выше.

In [52]:
```
patch_mask_values = []
patch_features = []

with torch.inference_mode():
    with torch.autocast(device_type='cuda', dtype=torch.float32):
        for image, mask in tqdm([(image_left, mask_left), (image_right, mask
            # processing mask
            mask = mask.split()[-1]
            mask_resized = resize_transform(mask)
            #mask_quantized = patch_quant_filter(mask_resized).squeeze().vie
            mask_quantized = patch_quant_filter(mask_resized).squeeze().deta
            patch_mask_values.append(mask_quantized)
            # processing image
            image = image.convert('RGB')
            image_resized = resize_transform(image)

            feats = get_features(image_resized)
            dim = feats[-1].shape[1]
            patch_features.append(feats[-1].squeeze().detach().cpu())
```

```
Processing images: 100%|██████████| 2/2 [00:00<00:00,  3.94it/s]
```

In [53]:
```
MASK_FG_THRESHOLD = 0.5

patch_features[0] = F.normalize(patch_features[0], p=2, dim=0)
patch_features[1] = F.normalize(patch_features[1], p=2, dim=0)

heatmaps = torch.einsum(
    "k f, f h w -> k h w",
    patch_features[0].view(dim, -1).permute(1, 0),
    patch_features[1],
)
```

```
# compute 2D patch locations in the left image
n_patches_left = patch_features[0].shape[1] * patch_features[0].shape[2]
patch_indices_left = torch.arange(n_patches_left)
locs_2d_left = (
    torch.stack(
        (
            patch_indices_left // patch_features[0].shape[2],  # row
            patch_indices_left % patch_features[0].shape[2]    # column
        ),
        dim=-1
    ) + 0.5
) * PATCH_SIZE

# compute the corresponding 2D patch locations in the right image
patch_indices_right = torch.flatten(heatmaps, start_dim=-2).argmax(dim=-1)
locs_2d_right = (
    torch.stack(
        (
            patch_indices_right // patch_features[1].shape[2],  # row
            patch_indices_right % patch_features[1].shape[2]    # column
        ),
        dim=-1
    ) + 0.5
) * PATCH_SIZE

# foreground patches mask in the left image
patches_left_fg_selection = (patch_mask_values[0].view(-1) > MASK_FG_THRESHO
# left image patches mask for patches that map to a foreground patch in the
patches_right_fg_selection = (patch_mask_values[1].view(-1)[patch_indices_ri
# select foreground left image patches that map to foreground right image pa
patches_fg_selection = (patches_left_fg_selection * patches_right_fg_selecti

# select (row, col) coordinates for the matched patches
locs_2d_left_fg = locs_2d_left[patches_fg_selection, :]
locs_2d_right_fg = locs_2d_right[patches_fg_selection, :]
```

Так как мы изменили правое изображение, мы можем не обучать PCA ещё раз, но сделаем это.

In [54]:
```
pca = PCA(n_components=3, whiten=True)
fg_patches_left = features_to_matrix(patch_features[0])[patches_fg_selectior
pca.fit(fg_patches_left)
```

Out[54]:

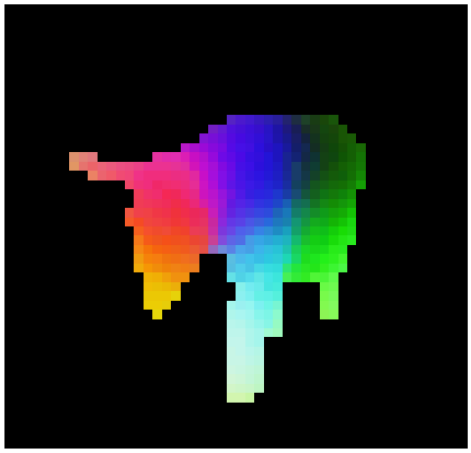
PCA

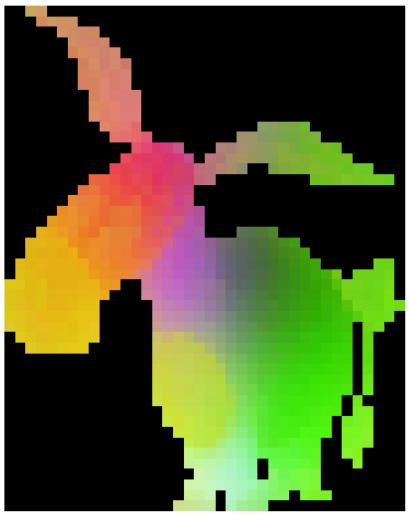
```
PCA(n_components=3, whiten=True)
```

In [55]:
```
projected_image_left = vis_dense_img(patch_features)
```

Изображение справа, Внутреннее представление

Изображение слева, Внутреннее представление





In [56]:
```
# image scale to go from patches to the original image coordinates
scale_left = image_left.height / IMAGE_SIZE
scale_right = image_right.height / IMAGE_SIZE

STRATIFY_DISTANCE_THRESHOLD = 100.0

print(f"Non-stratified points: {tuple(locs_2d_left_fg.shape)}")

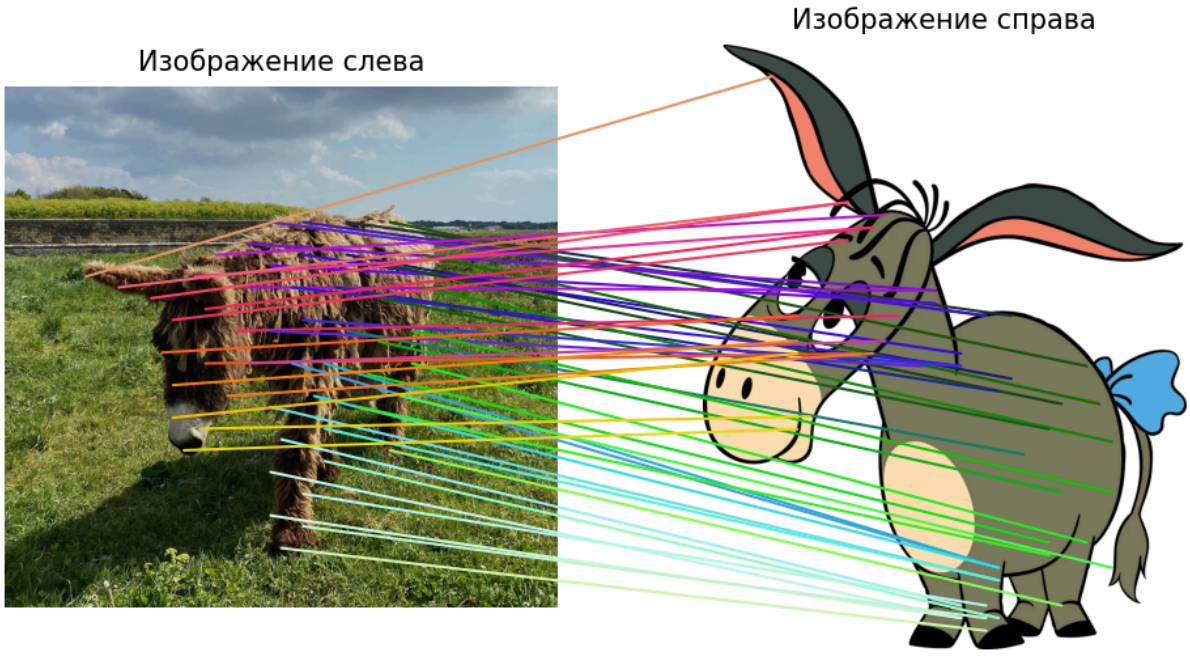
indices_to_exclude, indices_to_keep = stratify_points(locs_2d_left_fg * scal

sparse_points_left_yx = locs_2d_left_fg[indices_to_keep, :].cpu().numpy()
sparse_points_right_yx = locs_2d_right_fg[indices_to_keep, :].cpu().numpy()

print(f"Stratified points: {sparse_points_left_yx.shape}")
```

```
Non-stratified points: (484, 2)
Stratified points: (53, 2)
```

In [57]:
```
show_points(image_left, image_right, indices_to_keep, sparse_points_left_yx,
```



In [58]:
```
# image scale to go from patches to the original image coordinates
scale_left = image_left.height / IMAGE_SIZE
scale_right = image_right.height / IMAGE_SIZE

STRATIFY_DISTANCE_THRESHOLD = 200.0

print(f"Non-stratified points: {tuple(locs_2d_left_fg.shape)}")

indices_to_exclude, indices_to_keep = stratify_points(locs_2d_left_fg * scal

sparse_points_left_yx = locs_2d_left_fg[indices_to_keep, :].cpu().numpy()
sparse_points_right_yx = locs_2d_right_fg[indices_to_keep, :].cpu().numpy()

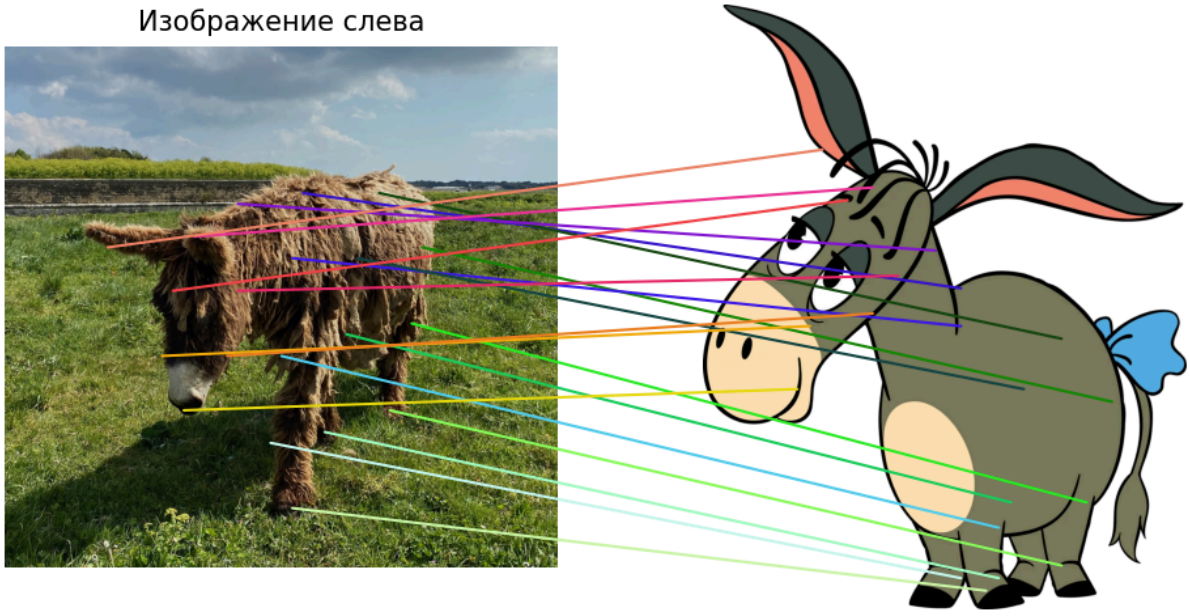
print(f"Stratified points: {sparse_points_left_yx.shape}")
```

```
Non-stratified points: (484, 2)
Stratified points: (20, 2)
```

In [59]:
```
show_points(image_left, image_right, indices_to_keep, sparse_points_left_yx,
```

Изображение справа

Изображение слева



Иные задачи

- Segmentation tracking Источник
- Depth estimation Источник
- Детекция Источник, Пример